Trading Client Documentation

github repository

Overview

A C++ trading client that connects to the Deribit API. The client supports both REST API calls for trading operations and WebSocket connections for real-time market data streaming.

Main Components

1. APIClient (APIClient.h, APIClient.cpp)

Handles REST API communications with Deribit.

Key features:
- Authentication with client credentials
- Token management with auto-refresh
- Trading operations (buy, sell, modify, cancel)
- Position and order book queries

Main methods:
- authenticate(): Get access token
- place_order(): Create new orders
- modify_order(): Change existing orders
- cancel_order(): Cancel open orders
- get_position(): Get current positions
- get_order_book(): Get market order book

2. WebSocketServer (WebSocketServer.h, WebSocketServer.cpp)

Manages WebSocket connections for real-time data.

Key features:
- SSL/TLS secure connections
- Multi-channel subscriptions
- Background thread for data streaming
- Data output to files

Main methods:
- start(): Begin streaming with given channels
- stop(): End streaming
- routine(): Background thread handler

3. Utils (utils.h, utils.cpp)

Helper functions for the application.

Key functions:
- get_client_info(): Get API credentials
- get_endpoints_info(): Load API endpoints
- get_env(): Read environment variables
- show_menu(): Display user interface

Setup Requirements

Environment Variables

Required:
- DEFAUL_CLIENT_ID: Deribit API client ID
- DEFAUL_CLIENT_SECRET: Deribit API client secret

Optional:
- CLIENT_ID: Temporary Deribit API client ID
- CLIENT_SECRET: Temporary Deribit API client secret

If CLIENT_ID and CLIENT_SECRET are set, they will override DEFAULT_CLIENT_ID and DEFAULT_CLIENT_SECRET.

Optional:
- SITE_URL: API base URL (default: https://test.deribit.com)
- Various endpoint paths (defaults to standard Deribit endpoints)

Dependencies

- libcurl: HTTP requests
- Boost: WebSocket handling
- nlohmann/json: JSON parsing

User Interface

The program offers a menu-driven interface with these options:

- ➔ 1. Place orders
- ➔ 2. Cancel orders
- ➔ 3. Modify orders
- ➔ 4. View order book
- ➔ 5. Check positions
- ➔ 6. Manage WebSocket channels
- ➔ 7. Start data streaming
- ➔ 8. Stop data streaming
- ➔ 9. Show menu
- ➔ 0. Exit

Error Handling

- Input validation for all user commands
- API error response handling
- WebSocket connection error management
- Environment variable fallbacks
- File operation error checks

Data Flow

1. User selects operation from menu
2. Program collects needed parameters
3. APIClient or WebSocketServer handles the request
4. Results are displayed or saved to file
5. Program returns to menu for next command

Threading

- Main thread: User interface and REST API calls
- WebSocket thread: Market data streaming